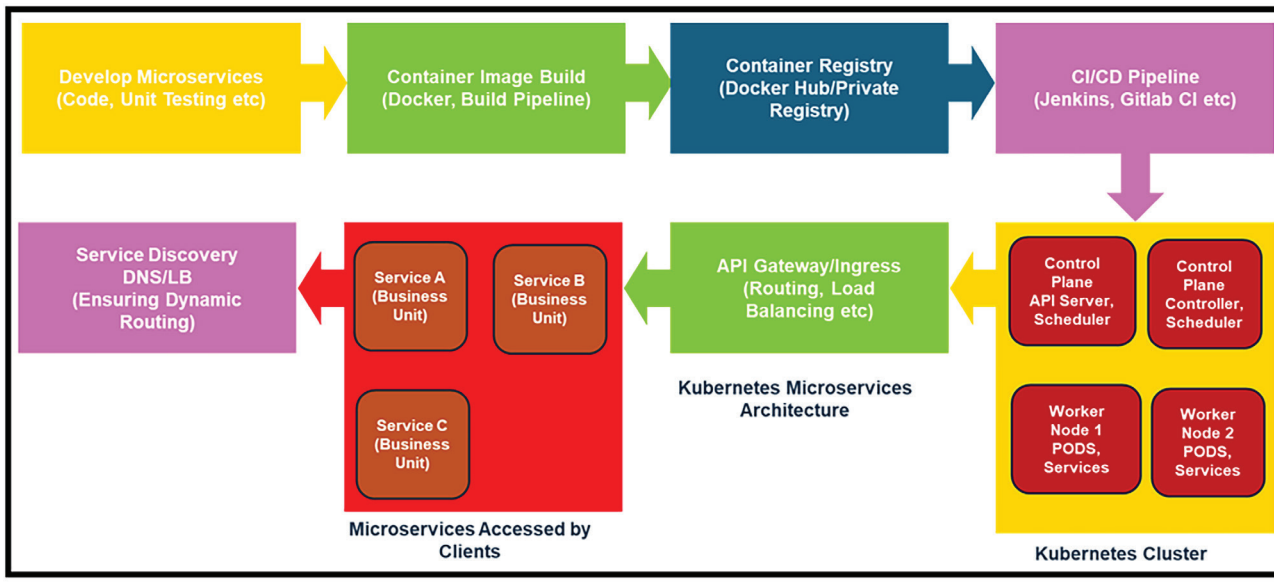

Figure 3: End-to-end flow of microservices deployment

dependencies), with a version for each image being maintained. Implementation of signature/digest for images enhances trust and reliability.

Monitoring: Microservices consist of many mobile parts; it is therefore all the more important that you measure everything efficiently and simply with mechanisms such as response time notifications, service error notifications and dashboards. Use logs such as Splunk and monitoring tools like AppDynamics.

Leverage built-in Kubernetes tools: Use commands like 'kubectl logs', 'kubectl describe', and 'kubectl get events' to troubleshoot the misbehaving of pods. These commands deliver critical insights into container logs, pod state, and recent cluster events, helping pinpoint issues quickly.

Centralised logging and monitoring: Implement a centralised logging system such as Elasticsearch/Kibana, Fluentd, or Loki to aggregate logs across the cluster. Similarly, monitoring tools like Prometheus, Grafana, or Kubernetes-native solutions can help visualise performance metrics, resource usage, and alert you to anomalous behaviour in real time.

Distributed tracing and debugging: Employ debugging and observability tools specifically designed for Kubernetes to provide insights into service mesh layers, ingress controllers, or cluster networking patterns.

Developers must decide early whether a microservices or monolithic architecture aligns best with their application's long-term scalability and usability goals. Choosing microservices from the outset can be more advantageous when significant expansion and frequent updates are anticipated.

Containers, Kubernetes, and microservices embody a collaborative evolution. The agility of microservices pairs with the efficient delivery of containers. Kubernetes provides robust orchestration and can scale individual microservices independently based on demand.

Containerisation allows for consistent behaviour of microservices from local development to production, reducing environment-specific issues. Lightweight containers and Kubernetes' self-healing capabilities lead to lower MTTR, as failing services can be quickly restarted or replaced. The independent deployment model minimises risks during updates. One service can be updated without the need to redeploy the entire application, reducing downtime and complexity.

Developers should be familiar with new tools such as centralised logging, distributed tracing, and container monitoring to handle the increased complexity. END

Acknowledgement

The author would like to thank Tanay Srivastava, director, Tricon Solutions LLC, for giving the required time and support to write this article as part of technical services efforts.

By: Dr Gopala Krishna Behara

The author is an enterprise architect at Tricon Solutions LLC. He has around 28 years of experience in IT.

Disclaimer: *The views expressed in this article are those of the author. Tricon Solutions LLC does not subscribe to the substance, veracity or truthfulness of this opinion.*